

El Problema del Estado Global

Objetivos

- Comprender las limitaciones del estado local
 - Identificar el problema del prop drilling
 - Reconocer cuándo necesitas estado global
 - Conocer las soluciones disponibles
-

1. Estado Local vs Estado Global



Estado Local

```
// =====
// QUÉ: Estado que vive dentro de un componente
// PARA: Datos que solo ese componente necesita
// IMPACTO: Simple, predecible, encapsulado
// =====

const Counter: React.FC = () => {
  // Estado local - solo Counter lo usa
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Contador: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
};
```

Estado Global

```
// =====
// QUÉ: Estado compartido entre múltiples componentes
// PARA: Datos que varios componentes necesitan
// IMPACTO: Sincronización automática, pero más complejo
// =====

// Ejemplos de estado global:
// - Usuario autenticado (Header, Sidebar, ProfilePage, etc.)
// - Carrito de compras (Navbar, CartPage, ProductCard, etc.)
// - Tema (dark/light) (App, todos los componentes)
// - Preferencias de idioma (toda la app)
```

2. El Problema: Prop Drilling

¿Qué es Prop Drilling?

Pasar props a través de múltiples niveles de componentes que no los necesitan, solo para llegar a un componente hijo que sí los usa.

```
// =====
// QUÉ: Props pasadas a través de componentes intermedios
// PARA: Llegar a un componente profundo en el árbol
// IMPACTO: Código verboso, difícil de mantener
// =====

// ❌ PROBLEMA: Prop Drilling
const App: React.FC = () => {
  const [user, setUser] = useState<User | null>(null);
  const [cartItems, setCartItems] = useState<CartItem[]>([]);

  return (
    <Layout
      user={user}
      cartItems={cartItems}
      setCartItems={setCartItems}>
      <MainContent
        user={user}
        cartItems={cartItems}
        setCartItems={setCartItems}
      />
    </Layout>
  );
};

// Layout no usa user ni cartItems, solo los pasa
const Layout: React.FC<LayoutProps> = ({
  user,
  cartItems,
  setCartItems,
  children,
}) => {
  return (
    <div>
      <Header
        user={user}
        cartItemsCount={cartItems.length}
      />
      <Sidebar user={user} />
      {children}
    </div>
  );
};

// MainContent tampoco los usa directamente
const MainContent: React.FC<MainContentProps> = ({
  user,
  cartItems,
  setCartItems,
}) => {
  return (
    <main>
      <ProductList
```

```

        user={user}
        cartItems={cartItems}
        setCartItems={setCartItems}
      />
    </main>
  );
};

// Finalmente, ProductCard sí los usa
const ProductCard: React.FC<ProductCardProps> = ({
  product,
  cartItems,
  setCartItems,
}) => {
  const addToCart = () => {
    setCartItems([...cartItems, { product, quantity: 1 }]);
  };

  return (
    <div>
      <h3>{product.name}</h3>
      <button onClick={addToCart}>Agregar al carrito</button>
    </div>
  );
};

```

Problemas del Prop Drilling

Problema	Descripción
Verbosidad	Muchas props repetidas en cada nivel
Acoplamiento	Componentes intermedios conocen datos que no usan
Mantenimiento	Agregar una prop requiere modificar toda la cadena
Refactoring	Mover componentes es complejo
Testing	Difícil testear componentes aislados

3. Solución con Context API

React incluye Context API para evitar prop drilling:

```

// =====
// QUÉ: Context API de React para estado compartido
// PARA: Evitar prop drilling básico
// IMPACTO: Solución nativa, pero con limitaciones
// =====

// Crear contexto
interface CartContextType {
  items: CartItem[];
  addItem: (item: CartItem) => void;
  removeItem: (id: number) => void;
  total: number;
}

```

```

}

const CartContext = createContext<CartItem | null>(null);

// Provider
const CartProvider: React.FC<{ children: React.ReactNode }> = ({
  children,
}) => {
  const [items, setItems] = useState<CartItem[]>([]);

  const addItem = (item: CartItem) => {
    setItems((prev) => [...prev, item]);
  };

  const removeItem = (id: number) => {
    setItems((prev) => prev.filter((item) => item.id !== id));
  };

  const total = items.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0,
  );

  return (
    <CartContext.Provider value={{ items, addItem, removeItem, total }}>
      {children}
    </CartContext.Provider>
  );
};

// Hook para usar el contexto
const useCart = () => {
  const context = useContext(CartContext);
  if (!context) {
    throw new Error('useCart debe usarse dentro de CartProvider');
  }
  return context;
};

// Uso en componente (sin prop drilling)
const ProductCard: React.FC<{ product: Product }> = ({ product }) => {
  const { addItem } = useCart();

  return (
    <div>
      <h3>{product.name}</h3>
      <button onClick={() => addItem({ ...product, quantity: 1 })}>
        Agregar
      </button>
    </div>
  );
};

```

Limitaciones de Context API

Limitación	Problema
------------	----------

Re-renders	Todos los consumidores se re-renderizan cuando el contexto cambia
Performance	No tiene optimización de selectores built-in
Boilerplate	Mucho código para cada contexto
DevTools	Sin herramientas de debugging dedicadas
Middleware	No soporta middleware nativamente

4. ¿Cuándo Necesitas Estado Global?

✓ Usar Estado Global

- **Datos de usuario** autenticado
- **Carrito de compras** en e-commerce
- **Tema** (dark/light mode)
- **Preferencias** de usuario
- **Notificaciones** globales
- **Estado de UI** compartido (modales, sidebars)

✗ NO Usar Estado Global

- Estado de **formularios** (mejor React Hook Form)
- Estado de **un solo componente**
- **Datos del servidor** (mejor React Query)
- Estado que **solo padres e hijos directos** comparten

5. Soluciones de Estado Global

Librería	Tamaño	Complejidad	Uso Ideal
Zustand	~1KB	Baja	Apps pequeñas-medianas
Redux Toolkit	~10KB	Media	Apps grandes, equipos grandes
Jotai	~2KB	Baja	Estado atómico
Recoil	~20KB	Media	Apps complejas con Facebook patterns
MobX	~15KB	Media	Programación reactiva

¿Por qué Zustand?

```
// =====
// QUÉ: Zustand - librería minimalista de estado
// PARA: Estado global simple, tipado, performante
// IMPACTO: Menos boilerplate que Redux, más features que Context
// =====

// Comparación: Context vs Zustand

// ✗ Context: ~40 líneas para un store básico
// ✓ Zustand: ~10 líneas para lo mismo

import { create } from 'zustand';
```

```
interface CounterStore {
  count: number;
  increment: () => void;
  decrement: () => void;
}

const useCounterStore = create<CounterStore>((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
  decrement: () => set((state) => ({ count: state.count - 1 })),
}));

// Uso
const Counter: React.FC = () => {
  const count = useCounterStore((state) => state.count);
  const increment = useCounterStore((state) => state.increment);

  return <button onClick={increment}>Count: {count}</button>;
};
```

Recursos Adicionales

- [React Docs - Context](#)
- [Zustand GitHub](#)
- [State Management Comparison](#)

Checklist de Comprensión

- ☐ Entiendo la diferencia entre estado local y global
- ☐ Puedo identificar el problema de prop drilling
- ☐ Conozco las limitaciones de Context API
- ☐ Sé cuándo usar estado global
- ☐ Entiendo por qué elegir Zustand

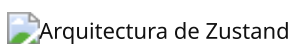
Siguiente: [02-introduccion-zustand.md](#)

Introducción a Zustand

Objetivos

- Instalar y configurar Zustand
- Crear tu primer store
- Entender la API básica de Zustand
- Usar el store en componentes React

1. ¿Qué es Zustand?



Zustand (alemán para "estado") es una librería de gestión de estado minimalista para React.

Características Principales

Característica	Descripción
Minimalista	~1KB, sin boilerplate
TypeScript	Soporte completo de tipos
Sin Provider	No necesita envolver la app
Selectores	Re-renders optimizados
Middleware	Persist, devtools, immer, etc.
Fuera de React	Funciona sin componentes

2. Instalación

```
# Con pnpm (recomendado)
pnpm add zustand
```

```
# Con yarn
yarn add zustand
```

3. Primer Store

```
// =====
// QUÉ: Crear un store con Zustand
// PARA: Centralizar estado y acciones
// IMPACTO: Estado accesible desde cualquier componente
// =====

// src/stores/counterStore.ts
import { create } from 'zustand';

// 1. Definir la interface del store
interface CounterStore {
  // Estado
  count: number;

  // Acciones
  increment: () => void;
  decrement: () => void;
  reset: () => void;
  setCount: (value: number) => void;
}

// 2. Crear el store con create()
const useCounterStore = create<CounterStore>((set) => ({
  // Estado inicial
  count: 0,

  // Acciones que modifican el estado
  increment: () => set((state) => ({ count: state.count + 1 })),
```

```

    decrement: () => set((state) => ({ count: state.count - 1 })),
    reset: () => set({ count: 0 }),
    setCount: (value) => set({ count: value }),
  }));

export { useCounterStore };

```

Anatomía de `create()`

```

// =====
// QUÉ: La función create() de Zustand
// PARA: Entender cómo funciona internamente
// IMPACTO: Mejor comprensión para stores complejos
// =====

const useStore = create<StoreType>((set, get, store) => ({
  // set: función para actualizar el estado
  // get: función para leer el estado actual
  // store: referencia al store completo (avanzado)

  value: 0,

  // Usar set para actualizar
  updateValue: (newValue) => set({ value: newValue }),

  // Usar set con función para acceder al estado previo
  increment: () => set((state) => ({ value: state.value + 1 })),

  // Usar get para leer el estado actual
  doubleValue: () => {
    const current = get().value;
    set({ value: current * 2 });
  },
}));

```

4. Usar el Store en Componentes

Método 1: Selector Individual

```

// =====
// QUÉ: Seleccionar valores específicos del store
// PARA: Evitar re-renders innecesarios
// IMPACTO: Componente solo se re-renderiza si ESE valor cambia
// =====

import { useCounterStore } from '../stores/counterStore';

const CounterDisplay: React.FC = () => {
  // Solo suscribirse a 'count'
  const count = useCounterStore((state) => state.count);

  console.log('CounterDisplay render'); // Solo cuando count cambia

```



```

    return <p>Contador: {count}</p>;
  };

const IncrementButton: React.FC = () => {
  // Solo suscribirse a 'increment'
  const increment = useCounterStore((state) => state.increment);

  console.log('IncrementButton render'); // Solo al montar

  return <button onClick={increment}>+1</button>;
};

```

Método 2: Múltiples Selectores

```

// =====
// QUÉ: Seleccionar múltiples valores
// PARA: Componentes que necesitan varios datos
// IMPACTO: Re-render cuando cualquiera de los valores cambia
// =====

const CounterControls: React.FC = () => {
  // Opción A: Selectores individuales
  const count = useCounterStore((state) => state.count);
  const increment = useCounterStore((state) => state.increment);
  const decrement = useCounterStore((state) => state.decrement);

  return (
    <div>
      <button onClick={decrement}>-</button>
      <span>{count}</span>
      <button onClick={increment}>+</button>
    </div>
  );
};

```

Método 3: Selector de Objeto (con cuidado)

```

// =====
// QUÉ: Seleccionar múltiples valores como objeto
// PARA: Conveniencia cuando necesitas varios valores
// IMPACTO: Requiere shallow para evitar re-renders
// =====

import { shallow } from 'zustand/shallow';

const CounterWithControls: React.FC = () => {
  // ⚠ Sin shallow: re-render en CUALQUIER cambio del store
  // ✅ Con shallow: re-render solo si count, increment o decrement cambian
  const { count, increment, decrement } = useCounterStore(
    (state) => ({
      count: state.count,
      increment: state.increment,
      decrement: state.decrement,
    })
  );
};

```

```

    shallow, // Comparación superficial
  );

  return (
    <div>
      <button onClick={decrement}>-</button>
      <span>{count}</span>
      <button onClick={increment}>+</button>
    </div>
  );
};

```

5. Ejemplo Completo: Todo Store

```

// =====
// QUÉ: Store de tareas con CRUD completo
// PARA: Ejemplo realista de gestión de estado
// IMPACTO: Base para aplicaciones con listas
// =====

// src/stores/todoStore.ts
import { create } from 'zustand';

interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

interface TodoStore {
  todos: Todo[];
  addTodo: (text: string) => void;
  toggleTodo: (id: number) => void;
  removeTodo: (id: number) => void;
  clearCompleted: () => void;
}

const useTodoStore = create<TodoStore>((set) => ({
  todos: [],

  addTodo: (text) =>
    set((state) => ({
      todos: [
        ...state.todos,
        {
          id: Date.now(),
          text,
          completed: false,
        },
      ],
    })),

  toggleTodo: (id) =>
    set((state) => ({

```

```

    todos: state.todos.map((todo) =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo,
    ),
  )),
})),

removeTodo: (id) =>
  set((state) => ({
    todos: state.todos.filter((todo) => todo.id !== id),
  })),

clearCompleted: () =>
  set((state) => ({
    todos: state.todos.filter((todo) => !todo.completed),
  })),
}));

export { useTodoStore };
export type { Todo };

```

Componentes que Usan el Store

```

// src/components/ToDoList.tsx
import { useTodoStore } from '../stores/todoStore';

const ToDoList: React.FC = () => {
  const todos = useTodoStore((state) => state.todos);
  const toggleTodo = useTodoStore((state) => state.toggleTodo);
  const removeTodo = useTodoStore((state) => state.removeTodo);

  if (todos.length === 0) {
    return <p>No hay tareas</p>;
  }

  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>
          <input
            type="checkbox"
            checked={todo.completed}
            onChange={() => toggleTodo(todo.id)}
          />
          <span
            style={{
              textDecoration: todo.completed ? 'line-through' : 'none',
            }}>
            {todo.text}
          </span>
          <button onClick={() => removeTodo(todo.id)}>✖</button>
        </li>
      ))}
    </ul>
  );
};

```

```
// src/components/AddTodo.tsx
import { useState } from 'react';
import { useTodoStore } from '../stores/todoStore';

const AddTodo: React.FC = () => {
  const [text, setText] = useState('');
  const addTodo = useTodoStore((state) => state.addTodo);

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (text.trim()) {
      addTodo(text.trim());
      setText('');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Nueva tarea..."
      />
      <button type="submit">Agregar</button>
    </form>
  );
};

// src/components/ToDoStats.tsx
const TodoStats: React.FC = () => {
  const todos = useTodoStore((state) => state.todos);
  const clearCompleted = useTodoStore((state) => state.clearCompleted);

  const total = todos.length;
  const completed = todos.filter((t) => t.completed).length;
  const pending = total - completed;

  return (
    <div>
      <p>
        Total: {total} | Completadas: {completed} | Pendientes: {pending}
      </p>
      {completed > 0 && (
        <button onClick={clearCompleted}>Limpiar completadas</button>
      )}
    </div>
  );
};
```

6. Sin Provider

A diferencia de Context o Redux, Zustand **no necesita Provider**:

```
// =====
// QUÉ: Zustand funciona sin envolver la app
```

```
// PARA: Simplicidad, menos boilerplate
// IMPACTO: Importa el hook y úsalo directamente
// =====

// ❌ Context/Redux: necesita Provider
const App = () => (
  <Provider store={store}>
    <CartProvider>
      <AuthProvider>
        <App />
      </AuthProvider>
    </CartProvider>
  </Provider>
);

// ✅ Zustand: sin Provider
const App = () => (
  <div>
    <Header /> {/* Puede usar useCartStore */}
    <MainContent /> {/* Puede usar useCartStore */}
    <Footer /> {/* Puede usar useCartStore */}
  </div>
);
```

Recursos Adicionales

- [Zustand Documentation](#)
- [Zustand GitHub](#)
- [TypeScript Guide](#)

✅ Checklist de Comprensión

- ☐ Sé instalar Zustand
- ☐ Puedo crear un store básico con create()
- ☐ Entiendo set, get y la interface del store
- ☐ Sé usar selectores en componentes
- ☐ Entiendo por qué Zustand no necesita Provider

Siguiente: [03-stores-y-acciones.md](#)

Stores y Acciones Tipadas

Objetivos

- Estructurar stores complejos con TypeScript
- Crear acciones con payloads tipados
- Implementar acciones asíncronas
- Organizar stores con slices
- Usar get() para acceder al estado actual

1. Estructura de un Store Complejo

Estructura del Store

```
// =====
// QUÉ: Store con estado complejo y múltiples acciones
// PARA: Aplicaciones reales con entidades relacionadas
// IMPACTO: Código organizado y mantenible
// =====

// src/stores/productStore.ts
import { create } from 'zustand';

// Tipos de dominio
interface Product {
  id: number;
  name: string;
  price: number;
  category: string;
  stock: number;
}

interface ProductFilters {
  search: string;
  category: string | null;
  minPrice: number | null;
  maxPrice: number | null;
}

// Interface del store
interface ProductStore {
  // Estado
  products: Product[];
  filters: ProductFilters;
  isLoading: boolean;
  error: string | null;

  // Acciones CRUD
  addProduct: (product: Omit<Product, 'id'>) => void;
  updateProduct: (id: number, updates: Partial<Product>) => void;
  deleteProduct: (id: number) => void;

  // Acciones de filtros
  setSearchFilter: (search: string) => void;
  setCategoryFilter: (category: string | null) => void;
  setPriceRange: (min: number | null, max: number | null) => void;
  clearFilters: () => void;

  // Acciones asíncronas
  fetchProducts: () => Promise<void>;
}

// Estado inicial de filtros
const initialFilters: ProductFilters = {
  search: '',
  category: null,
  minPrice: null,
```

```

    maxPrice: null,
  };

// Crear el store
const useProductStore = create<ProductStore>((set, get) => ({
  // Estado inicial
  products: [],
  filters: initialFilters,
  isLoading: false,
  error: null,

  // Acciones CRUD
  addProduct: (productData) =>
    set((state) => ({
      products: [...state.products, { ...productData, id: Date.now() }],
    })),

  updateProduct: (id, updates) =>
    set((state) => ({
      products: state.products.map((product) =>
        product.id === id ? { ...product, ...updates } : product,
      ),
    })),

  deleteProduct: (id) =>
    set((state) => ({
      products: state.products.filter((product) => product.id !== id),
    })),

  // Acciones de filtros
  setSearchFilter: (search) =>
    set((state) => ({
      filters: { ...state.filters, search },
    })),

  setCategoryFilter: (category) =>
    set((state) => ({
      filters: { ...state.filters, category },
    })),

  setPriceRange: (minPrice, maxPrice) =>
    set((state) => ({
      filters: { ...state.filters, minPrice, maxPrice },
    })),

  clearFilters: () => set({ filters: initialFilters }),

  // Acción asíncrona
  fetchProducts: async () => {
    set({ isLoading: true, error: null });

    try {
      const response = await fetch('/api/products');
      if (!response.ok) throw new Error('Error al cargar productos');

      const products = await response.json();
    }
  }
});

```

```

    set({ products, isLoading: false });
  } catch (error) {
    set({
      error: error instanceof Error ? error.message : 'Error desconocido',
      isLoading: false,
    });
  }
},
});

export { useProductStore };
export type { Product, ProductFilters };

```

2. Acciones con Payload Tipado

```

// =====
// QUÉ: Acciones que reciben datos estructurados
// PARA: Validación de tipos en tiempo de compilación
// IMPACTO: Menos errores, mejor autocompletado
// =====

interface CartItem {
  productId: number;
  quantity: number;
  price: number;
}

interface CartStore {
  items: CartItem[];

  // Acción con payload simple
  addItem: (item: CartItem) => void;

  // Acción con múltiples parámetros
  updateQuantity: (productId: number, quantity: number) => void;

  // Acción con payload parcial
  updateItem: (productId: number, updates: Partial<CartItem>) => void;

  // Acción sin payload
  clearCart: () => void;
}

const useCartStore = create<CartStore>((set) => ({
  items: [],

  addItem: (item) =>
    set((state) => {
      // Verificar si ya existe
      const existingIndex = state.items.findIndex(
        (i) => i.productId === item.productId,
      );

      if (existingIndex !== -1) {

```



```

    // Actualizar cantidad
    const newItems = [...state.items];
    newItems[existingIndex].quantity += item.quantity;
    return { items: newItems };
  }

  // Agregar nuevo
  return { items: [...state.items, item] };
}),

updateQuantity: (productId, quantity) =>
  set((state) => ({
    items:
      quantity <= 0
        ? state.items.filter((i) => i.productId !== productId)
        : state.items.map((i) =>
            i.productId === productId ? { ...i, quantity } : i,
          ),
  })),

updateItem: (productId, updates) =>
  set((state) => ({
    items: state.items.map((i) =>
      i.productId === productId ? { ...i, ...updates } : i,
    ),
  })),

  clearCart: () => set({ items: [] }),
}));

```

3. Usando get() para Acciones Complejas

```

// =====
// QUÉ: get() accede al estado actual dentro de acciones
// PARA: Lógica que depende del estado actual
// IMPACTO: Acciones más inteligentes y complejas
// =====

interface AuthStore {
  user: User | null;
  token: string | null;
  isAuthenticated: boolean;

  login: (credentials: Credentials) => Promise<void>;
  logout: () => void;
  refreshToken: () => Promise<void>;
  updateProfile: (updates: Partial<User>) => Promise<void>;
}

const useAuthStore = create<AuthStore>((set, get) => ({
  user: null,
  token: null,
  isAuthenticated: false,

```

```

login: async (credentials) => {
  const response = await fetch('/api/login', {
    method: 'POST',
    body: JSON.stringify(credentials),
  });

  const { user, token } = await response.json();
  set({ user, token, isAuthenticated: true });
},

logout: () => {
  set({ user: null, token: null, isAuthenticated: false });
},

refreshToken: async () => {
  // Usar get() para acceder al token actual
  const currentToken = get().token;

  if (!currentToken) {
    throw new Error('No hay token para refrescar');
  }

  const response = await fetch('/api/refresh', {
    headers: { Authorization: `Bearer ${currentToken}` },
  });

  const { token } = await response.json();
  set({ token });
},

updateProfile: async (updates) => {
  // Usar get() para acceder al usuario y token actuales
  const { user, token } = get();

  if (!user || !token) {
    throw new Error('Usuario no autenticado');
  }

  const response = await fetch(`/api/users/${user.id}`, {
    method: 'PATCH',
    headers: {
      Authorization: `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(updates),
  });

  const updatedUser = await response.json();
  set({ user: updatedUser });
},
}));

```

4. Organización con Slices

Para stores grandes, puedes dividir en "slices":

```

// =====
// QUÉ: Dividir un store grande en módulos (slices)
// PARA: Mejor organización y mantenibilidad
// IMPACTO: Código modular, fácil de escalar
// =====

// src/stores/slices/userSlice.ts
import { StateCreator } from 'zustand';

interface User {
  id: number;
  name: string;
  email: string;
}

export interface UserSlice {
  user: User | null;
  setUser: (user: User | null) => void;
  updateUser: (updates: Partial<User>) => void;
}

export const createUserSlice: StateCreator<UserSlice> = (set) => ({
  user: null,
  setUser: (user) => set({ user }),
  updateUser: (updates) =>
    set((state) => ({
      user: state.user ? { ...state.user, ...updates } : null,
    })),
});

// src/stores/slices/cartSlice.ts
import { StateCreator } from 'zustand';

interface CartItem {
  id: number;
  name: string;
  price: number;
  quantity: number;
}

export interface CartSlice {
  cartItems: CartItem[];
  addToCart: (item: CartItem) => void;
  removeFromCart: (id: number) => void;
  clearCart: () => void;
}

export const createCartSlice: StateCreator<CartSlice> = (set) => ({
  cartItems: [],
  addToCart: (item) =>
    set((state) => ({
      cartItems: [...state.cartItems, item],
    })),
  removeFromCart: (id) =>
    set((state) => ({
      cartItems: state.cartItems.filter((item) => item.id !== id),
    })),
});

```

```

    })),
    clearCart: () => set({ cartItems: [] }),
  });

// src/stores/slices/uiSlice.ts
import { StateCreator } from 'zustand';

export interface UISlice {
  sidebarOpen: boolean;
  theme: 'light' | 'dark';
  toggleSidebar: () => void;
  setTheme: (theme: 'light' | 'dark') => void;
}

export const createUISlice: StateCreator<UISlice> = (set) => ({
  sidebarOpen: true,
  theme: 'light',
  toggleSidebar: () => set((state) => ({ sidebarOpen: !state.sidebarOpen })),
  setTheme: (theme) => set({ theme }),
});

// src/stores/appStore.ts - Combinar slices
import { create } from 'zustand';
import { createUserSlice, UserSlice } from './slices/userSlice';
import { createCartSlice, CartSlice } from './slices/cartSlice';
import { createUISlice, UISlice } from './slices/uiSlice';

type AppStore = UserSlice & CartSlice & UISlice;

const useAppStore = create<AppStore>()((...args) => ({
  ...createUserSlice(...args),
  ...createCartSlice(...args),
  ...createUISlice(...args),
}));

export { useAppStore };

```

5. Acciones que Acceden a Otros Slices

```

// =====
// QUÉ: Slices que interactúan entre sí
// PARA: Lógica que involucra múltiples partes del estado
// IMPACTO: Acciones coordinadas entre slices
// =====

import { StateCreator } from 'zustand';
import { UserSlice } from './userSlice';
import { CartSlice } from './cartSlice';

// Tipo combinado para acceder a otros slices
type CombinedSlices = UserSlice & CartSlice & CheckoutSlice;

export interface CheckoutSlice {
  isProcessing: boolean;
}

```

```

    processCheckout: () => Promise<void>;
  }

export const createCheckoutSlice: StateCreator<
  CombinedSlices, // Tipo del store completo
  [],
  [],
  CheckoutSlice // Tipo de este slice
> = (set, get) => ({
  isProcessing: false,

  processCheckout: async () => {
    set({ isProcessing: true });

    // Acceder a otros slices con get()
    const { user, cartItems, clearCart } = get();

    if (!user) {
      throw new Error('Debe iniciar sesión');
    }

    if (cartItems.length === 0) {
      throw new Error('El carrito está vacío');
    }

    try {
      await fetch('/api/checkout', {
        method: 'POST',
        body: JSON.stringify({
          userId: user.id,
          items: cartItems,
        }),
      });

      // Llamar acción de otro slice
      clearCart();
    } finally {
      set({ isProcessing: false });
    }
  },
});

```

6. Tipos Avanzados con Generics

```

// =====
// QUÉ: Store genérico reutilizable
// PARA: Crear stores para diferentes entidades
// IMPACTO: DRY - evitar duplicación de código
// =====

interface Entity {
  id: number;
}

```

```

interface CRUDStore<T extends Entity> {
  items: T[];
  selectedId: number | null;
  isLoading: boolean;

  setItems: (items: T[]) => void;
  addItem: (item: Omit<T, 'id'>) => void;
  updateItem: (id: number, updates: Partial<T>) => void;
  deleteItem: (id: number) => void;
  selectItem: (id: number | null) => void;
  getSelectedItem: () => T | undefined;
}

// Factory para crear stores CRUD
const createCRUDStore = <T extends Entity>() => {
  return create<CRUDStore<T>>((set, get) => ({
    items: [],
    selectedId: null,
    isLoading: false,

    setItems: (items) => set({ items }),

    addItem: (itemData) =>
      set((state) => ({
        items: [...state.items, { ...itemData, id: Date.now() } as T],
      })),

    updateItem: (id, updates) =>
      set((state) => ({
        items: state.items.map((item) =>
          item.id === id ? { ...item, ...updates } : item,
        ),
      })),

    deleteItem: (id) =>
      set((state) => ({
        items: state.items.filter((item) => item.id !== id),
        selectedId: state.selectedId === id ? null : state.selectedId,
      })),

    selectItem: (id) => set({ selectedId: id }),

    getSelectedItem: () => {
      const { items, selectedId } = get();
      return items.find((item) => item.id === selectedId);
    },
  }));
};

// Usar la factory
interface Book {
  id: number;
  title: string;
  author: string;
}

```

```
interface Medicine {
  id: number;
  name: string;
  price: number;
}

const useBookStore = createCRUDStore<Book>();
const useMedicineStore = createCRUDStore<Medicine>();
```

Recursos Adicionales

- [Zustand TypeScript Guide](#)
- [Zustand Slices Pattern](#)
- [Zustand Best Practices](#)

✓ Checklist de Comprensión

- ☐ Sé estructurar stores complejos con TypeScript
- ☐ Puedo crear acciones con payloads tipados
- ☐ Entiendo cuándo usar get() vs set()
- ☐ Sé organizar stores grandes con slices
- ☐ Puedo crear stores genéricos reutilizables

Siguiente: [04-selectores-y-performance.md](#)

Selectores y Performance

Objetivos

- Entender cómo Zustand maneja re-renders
- Crear selectores eficientes
- Usar shallow para comparación superficial
- Implementar selectores derivados
- Optimizar performance con memoización

1. El Problema de los Re-renders

 Selectores y Performance

Sin Selectores: Re-render Excesivo

```
// =====
// QUÉ: Suscripción completa al store
// PARA: Obtener todo el estado (anti-patrón)
// IMPACTO: Re-render en CUALQUIER cambio del store
// =====

// ✗ MAL: Suscribirse a todo el store
const BadComponent: React.FC = () => {
  // Se re-renderiza cuando CUALQUIER cosa cambia en el store
```

```

const store = useProductStore();

console.log('BadComponent render'); // Muchos renders innecesarios

return <p>Productos: {store.products.length}</p>;
};

```

Con Selectores: Re-renders Optimizados

```

// =====
// QUÉ: Suscripción selectiva a partes del store
// PARA: Re-render solo cuando ESO cambia
// IMPACTO: Mejor performance, menos renders
// =====

// ✅ BIEN: Selector específico
const GoodComponent: React.FC = () => {
  // Solo se re-renderiza cuando products.length cambia
  const productCount = useProductStore((state) => state.products.length);

  console.log('GoodComponent render'); // Solo cuando es necesario

  return <p>Productos: {productCount}</p>;
};

```

2. Tipos de Selectores

Selector Simple

```

// =====
// QUÉ: Selector que extrae un valor directo
// PARA: Acceder a una propiedad del estado
// IMPACTO: Re-render solo si esa propiedad cambia
// =====

const ProductCount: React.FC = () => {
  // Selector simple - extrae products
  const products = useProductStore((state) => state.products);

  return <p>{products.length} productos</p>;
};

const LoadingIndicator: React.FC = () => {
  // Selector simple - extrae isLoading
  const isLoading = useProductStore((state) => state.isLoading);

  if (!isLoading) return null;
  return <div className="spinner">Cargando...</div>;
};

```

Selector Derivado (Computado)


```
// =====
// QUÉ: Selector que calcula un valor derivado
// PARA: Valores que dependen de otros valores
// IMPACTO: Cálculo en cada render, usar con cuidado
// =====

const CartSummary: React.FC = () => {
  // Selector derivado - calcula el total
  const total = useCartStore((state) =>
    state.items.reduce((sum, item) => sum + item.price * item.quantity, 0),
  );

  const itemCount = useCartStore((state) =>
    state.items.reduce((sum, item) => sum + item.quantity, 0),
  );

  return (
    <div>
      <p>{itemCount} artículos</p>
      <p>Total: ${total.toFixed(2)}</p>
    </div>
  );
};
```

Selector con Filtrado

```
// =====
// QUÉ: Selector que filtra datos
// PARA: Obtener subconjuntos de datos
// IMPACTO: Crea nuevo array en cada render
// =====

const CompletedTodos: React.FC = () => {
  // ⚠ Crea nuevo array cada vez - puede causar re-renders
  const completedTodos = useTodoStore((state) =>
    state.todos.filter((todo) => todo.completed),
  );

  return (
    <ul>
      {completedTodos.map((todo) => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
};
```

3. Shallow Equality

El Problema con Objetos/Arrays

```
// =====
// QUÉ: Problema de referencia con objetos
```

```
// PARA: Entender por qué shallow es necesario
// IMPACTO: Sin shallow, siempre re-renderiza
// =====

// ❌ PROBLEMA: Objeto nuevo en cada selector call
const BadMultiSelect: React.FC = () => {
  // Esto crea un NUEVO objeto cada vez
  // {} !== {} siempre es true → re-render infinito
  const { name, email } = useUserStore((state) => ({
    name: state.user?.name,
    email: state.user?.email,
  }));

  return (
    <p>
      {name} - {email}
    </p>
  );
};
```

Solución con shallow

```
// =====
// QUÉ: shallow compara propiedades superficialmente
// PARA: Evitar re-renders cuando los valores son iguales
// IMPACTO: Comparación por valor, no por referencia
// =====

import { shallow } from 'zustand/shallow';

// ✅ SOLUCIÓN: Usar shallow para comparación
const GoodMultiSelect: React.FC = () => {
  const { name, email } = useUserStore(
    (state) => ({
      name: state.user?.name,
      email: state.user?.email,
    }),
    shallow, // Compara { name, email } superficialmente
  );

  return (
    <p>
      {name} - {email}
    </p>
  );
};

// ✅ ALTERNATIVA: Selectores separados (más simple)
const AltMultiSelect: React.FC = () => {
  const name = useUserStore((state) => state.user?.name);
  const email = useUserStore((state) => state.user?.email);

  return (
    <p>
      {name} - {email}
    </p>
  );
};
```

```
    </p>
  );
};
```

Cuándo Usar shallow

```
// =====
// QUÉ: Guía de cuándo usar shallow
// PARA: Decisiones correctas de optimización
// IMPACTO: Código limpio y performante
// =====

// ✅ NO necesitas shallow - valor primitivo
const count = useStore((state) => state.count);

// ✅ NO necesitas shallow - referencia estable (función)
const increment = useStore((state) => state.increment);

// ⚠️ NECESITAS shallow - objeto nuevo
const { a, b } = useStore((state) => ({ a: state.a, b: state.b }), shallow);

// ⚠️ NECESITAS shallow - array nuevo
const items = useStore((state) => state.items.filter((x) => x.active), shallow);

// ✅ ALTERNATIVA a shallow - selectores individuales
const a = useStore((state) => state.a);
const b = useStore((state) => state.b);
```

4. Selectores Memoizados

Con useMemo (en componente)

```
// =====
// QUÉ: Memoizar cálculos derivados costosos
// PARA: Evitar recálculos innecesarios
// IMPACTO: Mejor performance en listas grandes
// =====

import { useMemo } from 'react';
import { shallow } from 'zustand/shallow';

const FilteredProductList: React.FC = () => {
  const { products, filters } = useProductStore(
    (state) => ({
      products: state.products,
      filters: state.filters,
    }),
    shallow,
  );

  // Memoizar el filtrado
  const filteredProducts = useMemo(() => {
    console.log('Calculando productos filtrados...'); // Solo cuando cambia
```

```

return products.filter((product) => {
  if (
    filters.search &&
    !product.name.toLowerCase().includes(filters.search.toLowerCase())
  ) {
    return false;
  }
  if (filters.category && product.category !== filters.category) {
    return false;
  }
  if (filters.minPrice && product.price < filters.minPrice) {
    return false;
  }
  if (filters.maxPrice && product.price > filters.maxPrice) {
    return false;
  }
  return true;
});
}, [products, filters]);

return (
  <ul>
    {filteredProducts.map((product) => (
      <li key={product.id}>{product.name}</li>
    ))}
  </ul>
);
};

```

Selectores en el Store

```

// =====
// QUÉ: Definir selectores como parte del store
// PARA: Reutilizar lógica de selección
// IMPACTO: DRY, lógica centralizada
// =====

interface TodoStore {
  todos: Todo[];
  filter: 'all' | 'active' | 'completed';

  // Acciones
  addTodo: (text: string) => void;
  toggleTodo: (id: number) => void;
  setFilter: (filter: 'all' | 'active' | 'completed') => void;

  // Selectores (getters)
  getFilteredTodos: () => Todo[];
  getStats: () => { total: number; completed: number; active: number };
}

const useTodoStore = create<TodoStore>((set, get) => ({
  todos: [],
  filter: 'all',

```

```

addTodo: (text) =>
  set((state) => ({
    todos: [...state.todos, { id: Date.now(), text, completed: false }],
  })),

toggleTodo: (id) =>
  set((state) => ({
    todos: state.todos.map((t) =>
      t.id === id ? { ...t, completed: !t.completed } : t,
    ),
  })),

setFilter: (filter) => set({ filter }),

// Selector como función del store
getFilteredTodos: () => {
  const { todos, filter } = get();
  switch (filter) {
    case 'active':
      return todos.filter((t) => !t.completed);
    case 'completed':
      return todos.filter((t) => t.completed);
    default:
      return todos;
  }
},

getStats: () => {
  const todos = get().todos;
  const completed = todos.filter((t) => t.completed).length;
  return {
    total: todos.length,
    completed,
    active: todos.length - completed,
  };
},
}));

// Uso en componente
const TodoList: React.FC = () => {
  const getFilteredTodos = useTodoStore((state) => state.getFilteredTodos);
  const filteredTodos = getFilteredTodos();

  return (
    <ul>
      {filteredTodos.map((todo) => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
};

```

5. Patrones de Performance

Componentes Separados

```
// =====
// QUÉ: Separar componentes por suscripción
// PARA: Aislar re-renders
// IMPACTO: Solo el componente afectado se re-renderiza
// =====

// ❌ MAL: Un componente con todo
const BadCart: React.FC = () => {
  const items = useCartStore((state) => state.items);
  const total = useCartStore((state) =>
    state.items.reduce((sum, i) => sum + i.price * i.quantity, 0)
  );
  const addItem = useCartStore((state) => state.addItem);

  // Todo se re-renderiza cuando cualquier cosa cambia
  return (
    <div>
      <CartItems items={items} />
      <CartTotal total={total} />
      <AddItemButton onAdd={addItem} />
    </div>
  );
};

// ✅ BIEN: Componentes separados
const CartItems: React.FC = () => {
  const items = useCartStore((state) => state.items);
  return (/* render items */);
};

const CartTotal: React.FC = () => {
  const total = useCartStore((state) =>
    state.items.reduce((sum, i) => sum + i.price * i.quantity, 0)
  );
  return <p>Total: ${total}</p>;
};

const AddItemButton: React.FC = () => {
  const addItem = useCartStore((state) => state.addItem);
  return <button onClick={() => addItem(/* item */)}>Agregar</button>;
};

const GoodCart: React.FC = () => {
  return (
    <div>
      <CartItems />
      <CartTotal />
      <AddItemButton />
    </div>
  );
};
```

useShallow Hook (Zustand v4.4+)

```
// =====  
// QUÉ: Hook useShallow para selección múltiple  
// PARA: Sintaxis más limpia que shallow  
// IMPACTO: Mismo resultado, mejor DX  
// =====  
  
import { useShallow } from 'zustand/react/shallow';  
  
const UserProfile: React.FC = () => {  
  // useShallow es más limpio que el segundo parámetro  
  const { name, email, avatar } = useUserStore(  
    useShallow((state) => ({  
      name: state.name,  
      email: state.email,  
      avatar: state.avatar,  
    })),  
  );  
  
  return (  
    <div>  
      <img  
        src={avatar}  
        alt={name}  
      />  
      <h2>{name}</h2>  
      <p>{email}</p>  
    </div>  
  );  
};
```

6. Debugging de Re-renders

```
// =====  
// QUÉ: Herramientas para debuggear re-renders  
// PARA: Identificar problemas de performance  
// IMPACTO: Optimización informada  
// =====  
  
import { useEffect, useRef } from 'react';  
  
// Hook para contar renders  
const useRenderCount = (componentName: string) => {  
  const renderCount = useRef(0);  
  renderCount.current += 1;  
  
  useEffect(() => {  
    console.log(`${componentName} rendered ${renderCount.current} times`);  
  });  
  
  return renderCount.current;  
};
```

```
// Uso
const MyComponent: React.FC = () => {
  const renderCount = useRenderCount('MyComponent');
  const data = useStore((state) => state.data);

  return (
    <div>
      <p>Renders: {renderCount}</p>
      <p>Data: {data}</p>
    </div>
  );
};
```

Recursos Adicionales

- [Zustand - Selecting State](#)
- [React DevTools Profiler](#)
- [Why Did You Render](#)

Checklist de Comprensión

- ☐ Entiendo cómo los selectores previenen re-renders
- ☐ Sé cuándo usar shallow
- ☐ Puedo crear selectores derivados con useMemo
- ☐ Sé separar componentes para optimizar renders
- ☐ Puedo debuggear problemas de re-renders

Siguiente: [05-persistencia-y-middleware.md](#)

Persistencia y Middleware

Objetivos

- Persistir estado en localStorage/sessionStorage
- Configurar el middleware persist
- Crear middleware personalizado
- Usar devtools para debugging
- Combinar múltiples middleware

1. Middleware en Zustand

 Flujo de Middleware

Los middleware interceptan y modifican el comportamiento del store.

Middleware Disponibles

Middleware	Uso
persist	Guardar estado en storage

devtools	Integración con Redux DevTools
immer	Mutaciones inmutables con sintaxis mutable
subscribeWithSelector	Suscripciones granulares

2. Persist Middleware

Configuración Básica

```
// =====  
// QUÉ: Persistir estado en localStorage  
// PARA: Mantener datos entre recargas de página  
// IMPACTO: Estado sobrevive a refresh/cierre del navegador  
// =====  
  
import { create } from 'zustand';  
import { persist } from 'zustand/middleware';  
  
interface SettingsStore {  
  theme: 'light' | 'dark';  
  language: string;  
  notifications: boolean;  
  setTheme: (theme: 'light' | 'dark') => void;  
  setLanguage: (language: string) => void;  
  toggleNotifications: () => void;  
}  
  
const useSettingsStore = create<SettingsStore>()(  
  persist(  
    (set) => ({  
      theme: 'light',  
      language: 'es',  
      notifications: true,  
  
      setTheme: (theme) => set({ theme }),  
      setLanguage: (language) => set({ language }),  
      toggleNotifications: () =>  
        set((state) => ({  
          notifications: !state.notifications,  
        })),  
    })),  
    {  
      name: 'settings-storage', // Clave en localStorage  
    },  
  ),  
);  
  
export { useSettingsStore };
```

Configuración Avanzada

```
// =====  
// QUÉ: Opciones avanzadas de persist
```

```

// PARA: Control fino sobre qué y cómo persistir
// IMPACTO: Optimización y seguridad
// =====

import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';

interface UserStore {
  user: User | null;
  token: string | null;
  preferences: UserPreferences;
  tempData: string; // No queremos persistir esto

  setUser: (user: User | null) => void;
  setToken: (token: string | null) => void;
  updatePreferences: (prefs: Partial<UserPreferences>) => void;
}

const useUserStore = create<UserStore>()(
  persist(
    (set) => ({
      user: null,
      token: null,
      preferences: { theme: 'light', fontSize: 16 },
      tempData: '',

      setUser: (user) => set({ user }),
      setToken: (token) => set({ token }),
      updatePreferences: (prefs) =>
        set((state) => ({
          preferences: { ...state.preferences, ...prefs },
        })),
    }),
    {
      name: 'user-storage',

      // Usar sessionStorage en lugar de localStorage
      storage: createJSONStorage(() => sessionStorage),

      // Solo persistir ciertas propiedades
      partialize: (state) => ({
        user: state.user,
        token: state.token,
        preferences: state.preferences,
        // tempData NO se persiste
      }),

      // Versión para migraciones
      version: 1,

      // Migrar estado de versiones anteriores
      migrate: (persistedState, version) => {
        if (version === 0) {
          // Migrar de v0 a v1
          return {
            ...persistedState,

```

```

        preferences: {
            theme: 'light',
            fontSize: 16,
        },
    };
}
return persistedState as UserStore;
},

// Callback cuando se rehidrata el estado
onRehydrateStorage: () => (state, error) => {
    if (error) {
        console.error('Error al rehidratar:', error);
    } else {
        console.log('Estado rehidratado:', state);
    }
},
},
),
);

```

Persistencia Selectiva

```

// =====
// QUÉ: Persistir solo partes del estado
// PARA: Excluir datos sensibles o temporales
// IMPACTO: Storage optimizado, seguridad mejorada
// =====

interface CartStore {
    items: CartItem[];
    lastUpdated: Date;
    isCheckedOut: boolean; // No persistir - es temporal

    addItem: (item: CartItem) => void;
    clearCart: () => void;
}

const useCartStore = create<CartStore>()(
    persist(
        (set) => ({
            items: [],
            lastUpdated: new Date(),
            isCheckedOut: false,

            addItem: (item) =>
                set((state) => ({
                    items: [...state.items, item],
                    lastUpdated: new Date(),
                })),

            clearCart: () => set({ items: [], lastUpdated: new Date() }),
        }),
        {
            name: 'cart-storage',

```

```

    // Solo persistir items y lastUpdated
    partialize: (state) => ({
      items: state.items,
      lastUpdated: state.lastUpdated,
    }),
  },
),
);

```

3. DevTools Middleware

```

// =====
// QUÉ: Integración con Redux DevTools
// PARA: Debugging visual del estado
// IMPACTO: Ver cambios de estado, time-travel debugging
// =====

import { create } from 'zustand';
import { devtools } from 'zustand/middleware';

interface CounterStore {
  count: number;
  increment: () => void;
  decrement: () => void;
}

const useCounterStore = create<CounterStore>()(
  devtools(
    (set) => ({
      count: 0,

      // El tercer parámetro de set es el nombre de la acción
      increment: () =>
        set(
          (state) => ({ count: state.count + 1 }),
          false, // replace = false (merge)
          'increment', // nombre de la acción en DevTools
        ),

      decrement: () =>
        set((state) => ({ count: state.count - 1 }), false, 'decrement'),
    }),
    {
      name: 'counter-store', // Nombre en DevTools
      enabled: process.env.NODE_ENV === 'development', // Solo en dev
    },
  ),
);

```

4. Combinar Middleware

```

// =====
// QUÉ: Usar múltiples middleware juntos
// PARA: Persist + DevTools + más
// IMPACTO: Funcionalidad completa del store
// =====

import { create } from 'zustand';
import { persist, devtools } from 'zustand/middleware';

interface TodoStore {
  todos: Todo[];
  addTodo: (text: string) => void;
  toggleTodo: (id: number) => void;
  removeTodo: (id: number) => void;
}

// El orden importa: devtools debe estar afuera de persist
const useTodoStore = create<TodoStore>()(
  devtools(
    persist(
      (set) => ({
        todos: [],

        addTodo: (text) =>
          set(
            (state) => ({
              todos: [
                ...state.todos,
                { id: Date.now(), text, completed: false },
              ],
            }),
            false,
            'addTodo',
          ),

        toggleTodo: (id) =>
          set(
            (state) => ({
              todos: state.todos.map((t) =>
                t.id === id ? { ...t, completed: !t.completed } : t,
              ),
            }),
            false,
            'toggleTodo',
          ),

        removeTodo: (id) =>
          set(
            (state) => ({
              todos: state.todos.filter((t) => t.id !== id),
            }),
            false,
            'removeTodo',
          ),
      })
    )
  ),
  {

```

```

        name: 'todo-storage',
      },
    ),
    {
      name: 'todo-store',
    },
  ),
);

```

5. Immer Middleware

```

// =====
// QUÉ: Escribir código mutable que produce estado inmutable
// PARA: Simplificar actualizaciones de estado anidado
// IMPACTO: Código más legible, menos errores
// =====

import { create } from 'zustand';
import { immer } from 'zustand/middleware/immer';

interface NestedStore {
  user: {
    profile: {
      name: string;
      address: {
        city: string;
        country: string;
      };
    };
    settings: {
      notifications: boolean;
      theme: string;
    };
  };
  updateCity: (city: string) => void;
  toggleNotifications: () => void;
}

const useNestedStore = create<NestedStore>()(
  immer((set) => ({
    user: {
      profile: {
        name: 'Juan',
        address: {
          city: 'Madrid',
          country: 'España',
        },
      },
      settings: {
        notifications: true,
        theme: 'light',
      },
    },
  }

```

```

// Sin immer: muy verboso
// updateCity: (city) => set((state) => ({
//   user: {
//     ...state.user,
//     profile: {
//       ...state.user.profile,
//       address: {
//         ...state.user.profile.address,
//         city,
//       },
//     },
//   },
// })),
// })),

// Con immer: simple y directo
updateCity: (city) =>
  set((state) => {
    state.user.profile.address.city = city;
  }),

toggleNotifications: () =>
  set((state) => {
    state.user.settings.notifications = !state.user.settings.notifications;
  }),
})),
);

```

6. Middleware Personalizado

```

// =====
// QUÉ: Crear middleware propio
// PARA: Logging, analytics, validación, etc.
// IMPACTO: Funcionalidad personalizada reutilizable
// =====

import { StateCreator, StoreMutatorIdentifier } from 'zustand';

// Tipo del middleware
type LoggerMiddleware = <
  T extends object,
  Mps extends [StoreMutatorIdentifier, unknown][] = [],
  Mcs extends [StoreMutatorIdentifier, unknown][] = [],
>({
  f: StateCreator<T, Mps, Mcs>,
  name?: string,
}) => StateCreator<T, Mps, Mcs>;

// Implementación del middleware
const logger: LoggerMiddleware = (config, name) => (set, get, api) =>
  config(
    (...args) => {
      const prevState = get();
      console.log(`[${name} || 'store']] Estado anterior:`, prevState);

```

```

        set(...args);

        const nextState = get();
        console.log(`[${name} || 'store']] Estado nuevo:`, nextState);
    },
    get,
    api,
  );

// Uso del middleware
const useLoggedStore = create<CounterStore>()(
  logger(
    (set) => ({
      count: 0,
      increment: () => set((state) => ({ count: state.count + 1 })),
      decrement: () => set((state) => ({ count: state.count - 1 })),
    }),
    'counter', // Nombre para los logs
  ),
);

```

Middleware de Validación

```

// =====
// QUÉ: Middleware que valida el estado
// PARA: Prevenir estados inválidos
// IMPACTO: Datos siempre consistentes
// =====

const withValidation =
  <T extends object>({
    config: StateCreator<T>,
    validate: (state: T) => boolean,
    errorMessage: string,
  }): StateCreator<T> =>
  (set, get, api) =>
    config(
      (partial, replace) => {
        const nextState =
          typeof partial === 'function'
            ? { ...get(), ...partial(get()) }
            : { ...get(), ...partial };

        if (!validate(nextState as T)) {
          console.error(errorMessage, nextState);
          return; // No aplicar el cambio
        }

        set(partial, replace);
      },
      get,
      api,
    );

// Uso

```



```

interface BalanceStore {
  balance: number;
  withdraw: (amount: number) => void;
  deposit: (amount: number) => void;
}

const useBalanceStore = create<BalanceStore>()(
  withValidation(
    (set) => ({
      balance: 100,
      withdraw: (amount) =>
        set((state) => ({ balance: state.balance - amount })),
      deposit: (amount) =>
        set((state) => ({ balance: state.balance + amount })),
    }),
    (state) => state.balance >= 0, // Validación
    'Error: El balance no puede ser negativo',
  ),
);

```

7. Rehidratación Asíncrona

```

// =====
// QUÉ: Manejar la carga inicial del estado persistido
// PARA: Mostrar loading mientras se carga el estado
// IMPACTO: UX suave sin parpadeos
// =====

import { create } from 'zustand';
import { persist } from 'zustand/middleware';

interface AuthStore {
  user: User | null;
  token: string | null;
  _hasHydrated: boolean;
  setHasHydrated: (state: boolean) => void;
}

const useAuthStore = create<AuthStore>()(
  persist(
    (set) => ({
      user: null,
      token: null,
      _hasHydrated: false,
      setHasHydrated: (state) => set({ _hasHydrated: state }),
    }),
    {
      name: 'auth-storage',
      onRehydrateStorage: () => (state) => {
        state?.setHasHydrated(true);
      },
    },
  ),
);

```

```
// Componente que espera la hidratación
const App: React.FC = () => {
  const hasHydrated = useAuthStore((state) => state._hasHydrated);

  if (!hasHydrated) {
    return <div>Cargando...</div>;
  }

  return <MainApp />;
};
```

Recursos Adicionales

- [Zustand Persist](#)
- [Zustand Middleware](#)
- [Redux DevTools](#)

Checklist de Comprensión

- ☐ Sé configurar persist básico
- ☐ Puedo usar partialize para persistencia selectiva
- ☐ Sé combinar devtools con persist
- ☐ Entiendo el middleware immer
- ☐ Puedo crear middleware personalizado

Final de la teoría · Semana 08

Ejercicio 01: Primer Store con Zustand

Objetivo

Crear tu primer store de Zustand con TypeScript, aprendiendo la sintaxis básica de `create()` y cómo usar el store en componentes React.

Instrucciones

Sigue los pasos en el archivo `starter/main.tsx`, descomentando el código progresivamente.

Conceptos Clave

- Función `create()` de Zustand
- Interface tipada para el store
- `set()` para actualizar estado
- Selectores en componentes

Tiempo Estimado

30-40 minutos

Recursos

- [Zustand - Getting Started](#)
 - [TypeScript Guide](#)
-

Ejercicio 02: Acciones Tipadas

Objetivo

Crear acciones con payloads tipados, implementar operaciones CRUD y usar `get()` para acceder al estado actual dentro de acciones.

Instrucciones

Sigue los pasos en el archivo `starter/todoStore.ts`, descomentando el código progresivamente.

Conceptos Clave

- Acciones con parámetros tipados
- Operaciones CRUD en el store
- Uso de `get()` para leer estado
- Acciones que dependen del estado actual

Tiempo Estimado

40-50 minutos

Recursos

- [Zustand - Actions](#)
 - [TypeScript - Utility Types](#)
-

Ejercicio 03: Selectores y Performance

Objetivo

Dominar el uso de selectores para optimizar re-renders, entender `shallow` para comparación superficial, y aplicar patrones de performance en Zustand.

Instrucciones

Sigue los pasos en el archivo `starter/selectorsDemo.tsx`, descomentando el código progresivamente.

Conceptos Clave

- Selectores simples vs derivados
- `shallow` para objetos/arrays
- `useShallow` hook
- Separación de componentes por suscripción
- Debugging de re-renders

Tiempo Estimado

40-50 minutos

Recursos

- [Zustand - Selecting State](#)
 - [React - useMemo](#)
-

Ejercicio 04: Persist Middleware

Objetivo

Implementar persistencia de estado en localStorage usando el middleware `persist` de Zustand, con configuración de opciones avanzadas.

Instrucciones

Sigue los pasos en el archivo `starter/persistStore.ts`, descomentando el código progresivamente.

Conceptos Clave

- Middleware `persist` básico
- `partialize` para persistencia selectiva
- `storage` personalizado (localStorage/sessionStorage)
- `version` y `migrate` para migraciones
- `onRehydrateStorage` callback

Tiempo Estimado

35-45 minutos

Recursos

- [Zustand - Persist](#)
 - [MDN - localStorage](#)
-

Ejercicio 05: Múltiples Stores

Objetivo

Organizar una aplicación con múltiples stores separados por dominio, aprender a comunicar stores entre sí y combinar con devtools.

Instrucciones

Sigue los pasos en el archivo `starter/multipleStores.ts`, descomentando el código progresivamente.

Conceptos Clave

- Separación de stores por dominio
- Comunicación entre stores
- Middleware devtools
- Combinación de `persist` + devtools

Tiempo Estimado

40-50 minutos

 Recursos

- [Zustand - Devtools](#)
- [Redux DevTools Extension](#)

 Proyecto Semana 08: Sistema de Gestión con Zustand

 Objetivo

Implementar un sistema de gestión completo usando Zustand para el estado global, aplicando stores múltiples, selectores optimizados, y persistencia.

 Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio único]

Ejemplos de Dominios

Dominio	Entidad Principal	Entidades Secundarias
 Biblioteca	Book	Loan, Member
 Farmacia	Medicine	Sale, Customer
 Gimnasio	Member	Attendance, Plan
 Escuela	Student	Course, Grade
 Restaurante	Dish	Order, Table
 Hospital	Patient	Appointment, Doctor
 Cine	Movie	Showtime, Ticket
 Hotel	Room	Booking, Guest

 Requisitos Funcionales

1. Arquitectura de Stores (Obligatorio)

Crear 3 stores separados:

Store Principal (entidad del dominio)

```
// Ejemplo: useBookStore, useMedicineStore, useMemberStore, etc.
interface MainStore {
  items: Item[];
  selectedId: number | null;
  filters: Filters;

  // CRUD
  addItem: (item:NewItem) => void;
  updateItem: (id: number, updates: Partial<Item>) => void;
  deleteItem: (id: number) => void;
```

```

// Selección
selectItem: (id: number | null) => void;

// Filtros
setFilter: (filter: Partial<Filters>) => void;
clearFilters: () => void;

// Getters
getFilteredItems: () => Item[];
getSelectedItem: () => Item | undefined;
}

```

Store de UI

```

interface UIStore {
  sidebarOpen: boolean;
  theme: 'light' | 'dark';
  activeModal: string | null;
  notifications: Notification[];

  toggleSidebar: () => void;
  setTheme: (theme: 'light' | 'dark') => void;
  openModal: (modalId: string) => void;
  closeModal: () => void;
  addNotification: (notification: Notification) => void;
  removeNotification: (id: number) => void;
}

```

Store de Usuario (Auth)

```

interface AuthStore {
  user: User | null;
  isAuthenticated: boolean;
  role: 'admin' | 'user' | null;

  login: (credentials: Credentials) => Promise<void>;
  logout: () => void;
  updateProfile: (updates: Partial<User>) => void;
}

```

2. Persistencia (Obligatorio)

- Store Principal: Persistir items en localStorage
- Store de UI: Persistir tema y sidebarOpen
- Store de Auth: Persistir user y token en sessionStorage

3. Selectores Optimizados (Obligatorio)

- Usar selectores individuales (no objetos completos)
- Implementar `shallow` donde sea necesario
- Usar `useMemo` para cálculos derivados costosos

4. Comunicación Entre Stores (Obligatorio)

- Al menos una acción que acceda a múltiples stores

- Ejemplo: "Procesar operación" que verifica auth, modifica items, y notifica UI

5. DevTools (Obligatorio)

- Integrar middleware devtools en todos los stores
- Nombrar acciones descriptivamente

Estructura del Proyecto

```
3-proyecto/
├─ README.md
├─ starter/
│  └─ src/
│     └─ stores/
│        ├── mainStore.ts      # Store principal del dominio
│        ├── uiStore.ts       # Store de UI
│        └─ authStore.ts      # Store de auth
│     └─ components/
│        ├── ItemList.tsx
│        ├── ItemCard.tsx
│        ├── ItemForm.tsx
│        ├── Header.tsx
│        ├── Sidebar.tsx
│        └─ Notifications.tsx
│     └─ hooks/
│        └─ useFilteredItems.ts
│     └─ types/
│        └─ index.ts
│     └─ App.tsx
│     └─ main.tsx
├─ index.html
├─ package.json
├─ tsconfig.json
└─ vite.config.ts
└─ solution/
   └─ ... (misma estructura)
```

Entregables

1. **3 stores funcionales** con TypeScript estricto
2. **Persistencia** configurada correctamente
3. **UI completa** con lista, formulario, filtros
4. **Comunicación** entre stores demostrada
5. **DevTools** funcionando
6. **README** con documentación de tu dominio

Criterios de Evaluación

Criterio	Puntos
Stores tipados correctamente	20
Persistencia funcional	15
Selectores optimizados	15

UI funcional completa	20
Comunicación entre stores	15
DevTools configurado	5
Código limpio y documentado	10
Total	100

Tips

1. **Empieza por los tipos:** Define interfaces antes de implementar
2. **Un store a la vez:** Completa cada store antes de pasar al siguiente
3. **Testea persistencia:** Recarga la página para verificar que los datos persisten
4. **Usa DevTools:** Redux DevTools es tu mejor amigo para debugging
5. **Selectores primero:** Piensa en qué datos necesita cada componente

Recursos

- [Zustand Docs](#)
- [Zustand Persist](#)
- [Redux DevTools](#)

Glosario - Semana 08: Gestión de Estado con Zustand

A

Action (Acción)

Función dentro del store que modifica el estado. En Zustand, las acciones se definen junto con el estado en el mismo objeto.

```
const useStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
}));
```

Atom (Átomo)

Unidad mínima de estado en librerías atómicas como Jotai. Zustand no usa este concepto, pero es útil conocerlo para comparativas.

B

Bear (Oso)

Mascota de Zustand. El nombre "Zustand" significa "estado" en alemán, y la librería usa un oso como logo (de ahí frases como "bear necessities").

Boilerplate

Código repetitivo necesario para configurar una funcionalidad. Zustand se caracteriza por tener muy poco boilerplate comparado con Redux.

C

create()

Función principal de Zustand para crear un store. Retorna un hook personalizado que los componentes usan para acceder al estado.

```
import { create } from 'zustand';
const useStore = create<State>()((set) => ({ ... }));
```

Currying

Técnica donde una función retorna otra función. `create()` en Zustand usa currying para el tipado de TypeScript:

```
create<State>()((set) => ({ ... }));
//           ↑↑ Doble paréntesis
```

D

Derived State (Estado Derivado)

Estado calculado a partir de otro estado. En Zustand se implementa con selectores o funciones dentro del store.

```
getFilteredItems: () => {
  const { items, filter } = get();
  return items.filter((item) => item.name.includes(filter));
};
```

Devtools

Middleware que conecta Zustand con Redux DevTools para debugging visual y time-travel.

```
import { devtools } from 'zustand/middleware';
create(devtools((set) => ({ ... })));
```

E

Ephemeral State (Estado Efímero)

Estado temporal que no necesita persistirse, como modales abiertos, tooltips, o estados de UI transitorios.

G

get()

Función proporcionada por Zustand para leer el estado actual dentro de acciones.

```
create((set, get) => ({
  items: [],
  addItem: (item) => {
    const current = get().items;
    set({ items: [...current, item] });
  }
}));
```

```
    },  
  }));
```

Global State (Estado Global)

Estado accesible desde cualquier parte de la aplicación, opuesto al estado local de un componente.

H

Hydration (Rehidratación)

Proceso de restaurar el estado desde storage (localStorage/sessionStorage) cuando la aplicación inicia.

Hook

Función de React que permite usar estado y otras características. Zustand crea hooks personalizados para cada store.

I

Immer

Librería que permite escribir código "mutable" que produce actualizaciones inmutables. Zustand tiene middleware nativo para Immer.

```
import { immer } from 'zustand/middleware/immer';
```

Immutability (Inmutabilidad)

Principio donde los datos no se modifican directamente, sino que se crean copias nuevas con los cambios.

L

Listener

Función que se ejecuta cuando el estado cambia. Zustand permite suscribirse a cambios:

```
useStore.subscribe((state) => console.log(state));
```

M

Middleware

Función que envuelve el store para agregar funcionalidad extra (persistencia, devtools, immer).

```
create(  
  devtools(  
    persist(  
      (set) => ({ ... }),  
      { name: 'store' }  
    )  
  )  
);
```

Migration (Migración)

Proceso de actualizar datos persistidos cuando la estructura del store cambia entre versiones.

```
persist(fn, {
  version: 2,
  migrate: (persistedState, version) => {
    if (version === 1) {
      // Migrar de v1 a v2
    }
    return persistedState;
  },
});
```

O

onRehydrateStorage

Callback que se ejecuta cuando finaliza la rehidratación del store desde storage.

```
persist(fn, {
  onRehydrateStorage: () => (state) => {
    console.log('Rehidratación completa', state);
  },
});
```

P

Partialize

Opción del middleware persist para guardar solo una parte del estado.

```
persist(fn, {
  partialize: (state) => ({
    items: state.items,
    // No persistir: isLoading, error, etc.
  }),
});
```

Persist

Middleware para guardar el estado en localStorage/sessionStorage automáticamente.

Provider

Componente contenedor necesario en Context/Redux. **Zustand NO requiere Provider**, lo cual simplifica el código.

R

Re-render

Cuando React vuelve a renderizar un componente. Los selectores de Zustand optimizan re-renders.

S

Selector

Función que extrae una parte específica del estado. Optimiza rendimiento evitando re-renders innecesarios.

```
// Selector simple
const count = useStore((state) => state.count);

// Selector con cálculo
const total = useStore((state) =>
  state.items.reduce((sum, item) => sum + item.price, 0),
);
```

set()

Función principal para actualizar el estado en Zustand.

```
set({ count: 10 }); // Reemplazar
set((state) => ({ count: state.count + 1 })); // Actualizar basado en estado anterior
set({ count: 10 }, true); // Reemplazar todo el estado
```

Shallow (Comparación Superficial)

Comparación que revisa solo el primer nivel de un objeto. Usado para optimizar selectores que retornan objetos.

```
import { shallow } from 'zustand/shallow';
const { name, email } = useStore(
  (state) => ({ name: state.name, email: state.email }),
  shallow,
);
```

Slice

Porción del estado con su lógica asociada. Patrón para organizar stores grandes.

```
const createUserSlice = (set) => ({
  user: null,
  setUser: (user) => set({ user }),
});

const createCartSlice = (set) => ({
  items: [],
  addItem: (item) => set((s) => ({ items: [...s.items, item] })),
});
```

Store

Contenedor centralizado del estado de la aplicación en Zustand.

Subscribe

Método para escuchar cambios en el estado fuera de componentes React.

```
const unsubscribe = useStore.subscribe(
  (state) => state.count,
  (count) => console.log('Count cambió a:', count),
);
```

T

Transient Update

Actualización de estado que no persiste ni dispara re-renders, útil para animaciones.

U

useShallow

Hook de Zustand (v4.4+) para comparación superficial automática.

```
import { useShallow } from 'zustand/react/shallow';
const { name, email } = useStore(
  useShallow((state) => ({ name: state.name, email: state.email })),
);
```

V

Vanilla Store

Store de Zustand que funciona fuera de React.

```
import { createStore } from 'zustand/vanilla';
const store = createStore((set) => ({ count: 0 }));
```

Z

Zustand

Librería de gestión de estado para React. Significa "estado" en alemán. Creada por el equipo pmndrs (Poimandres).

Tabla Comparativa Rápida

Término	Zustand	Redux	Context
Crear store	create()	createStore()	createContext()
Actualizar	set()	dispatch()	setState()
Leer estado	Selector	useSelector()	useContext()
Provider	❌ No necesario	✅ Requerido	✅ Requerido
Middleware	✅ Composable	✅ applyMiddleware	❌ No nativo
DevTools	✅ Middleware	✅ Nativo	❌ No nativo

Bundle size	~1.2KB	~4.5KB	0KB (nativo)
-------------	--------	--------	--------------

[← Volver a Semana 08](#)

Semana 08: Gestión de Estado Global con Zustand

Objetivos de Aprendizaje

Al finalizar esta semana, serás capaz de:

- ✓ Comprender el problema del estado global en aplicaciones React
- ✓ Implementar stores con Zustand y TypeScript
- ✓ Crear acciones y selectores tipados
- ✓ Persistir estado en localStorage
- ✓ Implementar middleware personalizado
- ✓ Aplicar patrones de organización de stores
- ✓ Integrar Zustand con React Router

Requisitos Previos

- Semana 07 completada (React Router v6)
- Conocimiento de hooks de React (useState, useEffect)
- TypeScript intermedio (genéricos, interfaces)

Estructura de la Semana

```
week-08/
├── README.md
├── rubrica-evaluacion.md
├── 0-assets/
│   ├── 01-state-management-problem.svg
│   ├── 02-zustand-architecture.svg
│   ├── 03-store-structure.svg
│   ├── 04-selectors-performance.svg
│   └── 05-middleware-flow.svg
├── 1-teoria/
│   ├── 01-problema-estado-global.md
│   ├── 02-introduccion-zustand.md
│   ├── 03-stores-y-acciones.md
│   ├── 04-selectores-y-performance.md
│   └── 05-persistencia-y-middleware.md
├── 2-ejercicios/
│   ├── ejercicio-01-primer-store/
│   ├── ejercicio-02-acciones-tipadas/
│   ├── ejercicio-03-selectores/
│   ├── ejercicio-04-persist-middleware/
│   └── ejercicio-05-multiples-stores/
├── 3-proyecto/
│   ├── README.md
│   ├── starter/
│   └── solution/
└── 4-recursos/
```

- └─ ebooks-free/
- └─ videografia/
- └─ webgrafia/
- └─ 5-glosario/
- └─ README.md

Contenidos

Teoría (2.5 horas)

#	Tema	Archivo	Duración
1	El Problema del Estado Global	01-problema-estado-global.md	25 min
2	Introducción a Zustand	02-introduccion-zustand.md	30 min
3	Stores y Acciones	03-stores-y-acciones.md	35 min
4	Selectores y Performance	04-selectores-y-performance.md	30 min
5	Persistencia y Middleware	05-persistencia-y-middleware.md	30 min

Ejercicios (3.5 horas)

#	Ejercicio	Carpeta	Duración
1	Primer Store	ejercicio-01-primer-store	35 min
2	Acciones Tipadas	ejercicio-02-acciones-tipadas	40 min
3	Selectores	ejercicio-03-selectores	45 min
4	Persist Middleware	ejercicio-04-persist-middleware	40 min
5	Múltiples Stores	ejercicio-05-multiples-stores	50 min

Proyecto (2 horas)

Proyecto	Descripción	Carpeta
App con Estado Global	Aplicación con múltiples stores integrados	3-proyecto

Distribución del Tiempo

Actividad	Tiempo
Teoría	2.5 horas
Ejercicios	3.5 horas
Proyecto	2 horas
Total	8 horas

Entregable

Proyecto Semanal 📦 (100%) - Aplicación con estado global usando Zustand

 **Nota:** La teoría y los ejercicios son recursos de aprendizaje para prepararte para el proyecto. No son entregables evaluados.

✓ Checklist de la Semana

Preparación (No evaluada)

- ☐ Leer material teórico (5 módulos)
- ☐ Completar ejercicios guiados (5 ejercicios)
- ☐ Revisar recursos adicionales si es necesario

Entregable (Evaluado - 100%)

- ☐ Proyecto semanal completado y funcional
- ☐ Código subido al repositorio
- ☐ README del proyecto actualizado

Navegación

← Anterior	Índice	Siguiente →
Semana 07: React Router	Bootcamp	Semana 09: Redux Toolkit

Semana 08 de 20 · Etapa 3: React Intermedio

Rúbrica de Evaluación - Semana 08

Gestión de Estado Global con Zustand

Recursos de Aprendizaje (No Evaluados)

La teoría y los ejercicios son materiales de preparación para el proyecto. No son entregables evaluados, pero son esenciales para desarrollar las competencias necesarias.

Conocimientos Esperados

- Comprensión del problema del estado global y prop drilling
- Arquitectura de Zustand: stores, acciones, selectores
- Patrones de organización de stores
- Selectores y optimización de performance
- Persistencia y middleware

Habilidades a Desarrollar

- Crear stores tipados con TypeScript
- Implementar acciones con payload tipado
- Usar selectores para optimización
- Aplicar middleware de persistencia
- Integrar Zustand con componentes React

Competencias a Evaluar

Proyecto (100%)

Funcionalidad (40%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
Features completas	Todas las features implementadas y funcionando	Features principales completas	Features básicas funcionando	Features incompletas
Store principal	Store completo con estado y acciones	Store funcional	Store básico	Store incompleto
Selectores	Selectores derivados con optimización	Selectores funcionales	Selectores básicos	Sin selectores
Persistencia	localStorage configurado y funcionando	Persistencia correcta	Persistencia básica	Sin persistencia
Integración con Router	Estado sincronizado con navegación	Integración correcta	Integración básica	Sin integración

Adaptación al Dominio (35%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
Coherencia del dominio	Entidades y acciones 100% contextualizadas	Buena adaptación al dominio	Adaptación básica	Sin adaptación
Originalidad	Implementación única y creativa	Implementación propia	Implementación base	Copia de otros aprendices
Entidades modeladas	Modelos completos y relacionados	Modelos correctos	Modelos básicos	Modelos incompletos

Calidad del Código (25%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
Tipado TypeScript	Tipos exhaustivos, sin any	Tipos correctos	Tipos parciales	Muchos any o errores
Organización	Stores modulares, bien separados	Estructura clara	Organización básica	Código desorganizado
Nombres y comentarios	Nombres descriptivos, comentarios claros	Buenos nombres	Nombres aceptables	Nombres confusos
Sin errores	Cero warnings y errores	Mínimos warnings	Algunos warnings	Errores en consola

 Distribución de la Evaluación

Componente	Peso
Funcionalidad	40%
Adaptación al Dominio	35%
Calidad del Código	25%
Total Proyecto	100%

Escala de Calificación

Rango	Calificación	Descripción
90-100%	Excelente	Dominio completo del tema
80-89%	Bueno	Buen manejo con detalles menores
70-79%	Suficiente	Cumple requisitos mínimos
<70%	Insuficiente	No alcanza el nivel requerido

Adaptación por Dominio

Cada aprendiz implementa el estado global para su dominio asignado:

Dominio	Store Principal	Entidades	Acciones Clave
 Biblioteca	useLibraryStore	Book, Loan, Member	addBook, borrowBook, returnBook
 Farmacia	usePharmacyStore	Medicine, Sale, Inventory	addMedicine, sell, updateStock
 Gimnasio	useGymStore	Member, Routine, Attendance	registerMember, checkIn, assignRoutine
 Escuela	useSchoolStore	Student, Course, Grade	enrollStudent, assignGrade, addCourse
 Restaurante	useRestaurantStore	Dish, Order, Table	addDish, takeOrder, closeTable
 Hospital	useHospitalStore	Patient, Appointment, Doctor	registerPatient, scheduleAppointment
 Cine	useCinemaStore	Movie, Showtime, Ticket	addMovie, sellTicket, scheduleShow
 Hotel	useHotelStore	Room, Reservation, Guest	bookRoom, checkIn, checkOut

Lista de Verificación del Proyecto

Requisitos Funcionales

- ☐ Store principal con estado del dominio
- ☐ Acciones CRUD para entidades principales
- ☐ Selectores derivados (filtros, totales, búsquedas)
- ☐ Persistencia en localStorage
- ☐ Integración con React Router
- ☐ Al menos 2 stores separados

Requisitos Técnicos

- ☐ TypeScript estricto (sin any)
- ☐ Stores tipados con interfaces
- ☐ Acciones con payload tipado
- ☐ Selectores con tipos de retorno
- ☐ Middleware persist configurado
- ☐ Componentes con suscripción selectiva

Requisitos de Calidad

- ☐ Código organizado en módulos

- ☐ Nombres descriptivos en inglés
- ☐ Comentarios en español (QUÉ/PARA/IMPACTO)
- ☐ Sin warnings ni errores
- ☐ Performance optimizada



Criterios de Aprobación

- **Mínimo 70%** en la calificación total del proyecto
- Proyecto funcional y tipado correctamente
- Implementación coherente con el dominio asignado
- Sin copia de otros aprendices (dominio único)



Formato de Entrega

Entregable Único: Proyecto Semanal

```
week-08/  
└─ 3-proyecto/  
    └─ mi-proyecto/  
        ├── src/  
        │   ├── stores/      # Stores de Zustand  
        │   ├── components/  # Componentes React  
        │   └─ ...  
        ├── package.json  
        └─ README.md        # Documentación del proyecto
```

Fecha límite: Según calendario del bootcamp

💡 Los ejercicios (carpeta `2-ejercicios/`) son para práctica personal y no requieren entrega.

Rúbrica Semana 08 · Zustand